title

Vittorio Porri

27 marzo 2026

Indice

Indice **3**

1 Euler Tour Trees **1**
1.0.1 Operazioni 2

Struttura dati **5**

2 Balanced Binary Search Trees **7**
2.1 Alberi Binari di Ricerca 7
2.1.1 Operazioni 7
2.2 Red-Black Trees 11

Bibliography **21**

Euler Tour Trees 1

1.0.1 Operazioni 2

In questa tesi esploreremo gli Euler Tour Trees (ETT), una struttura dati dinamica introdotta da Henzinger e King [1], progettata per mantenere informazioni su foreste soggette a modifiche strutturali.

L'obiettivo è supportare operazioni di:

- **link(u,v)**: aggiungere un arco (u, v) unendo due alberi.
- **cut(u,v)**: rimuovere l'arco (u, v) dividendo l'albero in due.
- **isConnected(u,v)**: verificare se u e v appartengono allo stesso albero.

A differenza di altre strutture che operano direttamente sui nodi, gli ETT si basano sulla linearizzazione dell'albero tramite cicli di Eulero. Questa rappresentazione permette di ridurre complessi problemi di connettività a semplici manipolazioni di sequenze.

Definition 1.0.1 *Un ciclo euleriano è un percorso in un grafo che inizia e finisce nello stesso nodo passando esattamente una volta per ogni arco.*

Note

Poiché gli alberi non hanno veri cicli euleriani, consideriamo ogni arco $e = (u, v)$ come una coppia di archi diretti (u, v) e (v, u).

L'idea della stuttura dati è quella di creare il ciclo euleriano visitando l'albero tramite una ricerca in profondità (DFS) tenendo traccia dell'ordine in cui vengono visitati i nodi

Algorithm 1: Creazione Tour Euleriano

Input: Nodo corrente u, Lista L
Output: Cammino euleriano in L

```
L ← u
if u.left ≠ NULL then
    makeTour(u.left, L)
    L ← u
end
if u.right ≠ NULL then
    makeTour(u.right, L)
    L ← u
end
```

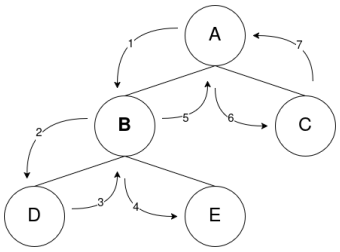

Figura 1.1: Il Ciclo euleriano che si forma è ABDBEBACA

Il ciclo euleriano è strettamente connesso con la struttra dell'albero, ad esempio si osserva che la sequanza A**BDBEB**ACA rappresenta il sottoalbero radicato in BACK

Claim

La sequenza di nodi visitati tra la prima e l'ultima istanza di un nodo v è il cammino euleriano del sottoalbero radicato in v

1.0.1 Operazioni

isConnected

L'operazione **isConnected(u,v)** è la più semplice poichè è sufficente verificare che i due nodi u e v si trovino nello stesso ciclo euleriano, per farlo basta verificare che il primo elemento della sequanza sia il medesimo per i due nodi

Rerooting

Prima di introdurre le altre operazioni introduciamo, un'operazione ausiliaria **rerooting(v)**, tale operazione ha il compito di rendere un nodo interno del ciclo euleriano il primo nodo del ciclo , per farlo seguiamo i seguenti passaggi:

1. si cerca la prima occorrenza del nodo v nel ciclo euleriano, e si separa la sequanza in A che conterrà tutto ciò che c'era prima di tale nodo e B che conterrà il nodo v e tutto ciò che c'è dopo
2. si elimina il primo carattere di A
3. riconcateno $B + A + B[0]$

Algorithm 2: Rerooting

Input: ETT L, nodo v
Output: il nuovo ETT

```
if L[0] == v then
    return L
end
A ← ∅
B ← ∅
isFound ← False
for i ← 0 to len(L) − 1 do
    if L[i] == v and ¬isFound then
        isFound ← True
    end
    if ¬isFound then
        A ← A + L[i]
    end
    else
        B ← B + L[i]
    end
end
return B + A + B[0]
```

Link

L'operazione **link(u,v)** permette di unire due alberi distinti aggiungendo un arco tra il nodo u (appartenente al primo albero) e il nodo v (appartenente al secondo), per farlo seguiamo i seguenti passaggi:

1. C = rerooting(v)
2. si cerca l'ultima occorrenza di v nel suo ciclo euleriano, e si separa la sequanza in A che conterrà tutto ciò che c'era prima di tale nodo e il nodo v e B che conterrà tutto ciò che c'è dopo
3. concateno $A + C + u + B$

Algorithm 3: Link

Input: ETT L, ETT C, nodo u, nodo v
Output: l'ETT del nuovo albero

```
C ← rerooting(C, v)
A ← ∅
B ← ∅
isFound ← False
for i ← len(L) − 1 to −1 do
    if L[i] == u and ¬isFound then
        isFound ← True
    end
    if ¬isFound then
        B ← L[i] + B
    end
    else
        A ← L[i] + A
    end
end
return A + C + u + B
```

Cut

L'operazione **cut(u,v)** permette di separare un albero in due alberi distinti, eliminando l'arco (u, v) crando due nuovi alberi radicati rispettivamente in u e v, per farlo seguiamo i seguenti passaggi:

1. rerooting(v)
2. si cerca la sequanza tra la prima e l'ultima occorrenza di u e si separea in A che conterrà tutto ciò che c'era prima della prima occorrenza di u, B che conterrà tutto ciò che c'è tra la prima e l'ultima occorrenza di u e C che contiene ciò che c'è dopo l'ultima occorrenza di u
3. si elimina l'ultimo carattere in A
4. si restituisce $A + C$ e B

Algorithm 4: Cut

Input: ETT L, nodo u, nodo v
Output: l'ETT L_1 e L_2 dei due nuovi alberi

```
L ← rerooting(L, v)
A ← ∅
B ← ∅
C ← ∅
// Trova la prima occorrenza di u
for i ← 0 to len(L) − 1 do
    if L[i] == u then
        break
    end
    A ← A + L[i]
end
elimino l'ultimo elemento di A
// Trova l'ultima occorrenza di u
for j ← len(L) − 1 to 0 do
    if L[j] == u then
        break
    end
    C ← L[j] + C
end
// Costruisce la sequenza B tra le due occorrenze
for k ← i to j do
    B ← B + L[k]
end
return (A + C), B
```

Remark 1.0.1 Tutte queste implementazioni delle operazioni però hanno costo proporzionale alla dimensione del ciclo euleriano dunque il loro costo sara $O(n)$

L'obbiettivo quindi sara quello di introdurre una struttra dati che ci permetterà di eseguire queste operazioni in maniera molto piu efficente

Struttura dati

Balanced Binary Search Trees 2

2.1 Alberi Binari di Ricerca . 7
2.1.1 Operazioni 7
2.2 Red-Black Trees 11

2.1 Alberi Binari di Ricerca

Un Albero Binario di Ricerca (BST) è una struttura dati ad albero che soddisfa le seguenti proprietà per ogni nodo v:

- Ogni nodo contiene un elemento a cui è associata una chiave presa da un dominio totalmente ordinato.
- Tutte le chiavi nel sottoalbero **sinistro** di v sono minori o uguali ($\leq$) alla chiave in v.
- Tutte le chiavi nel sottoalbero **destro** di v sono maggiori o uguali ($\geq$) alla chiave in v.

Note

Eseguendo una visita **simmetrica** dei nodi, le chiavi vengono enumerate in ordine crescente.

Dimostrazione. La dimostrazione procede per induzione sull'altezza h dell'albero:

- **Caso base** $h = 1$: L'albero è composto dalla sola radice e dai due figli foglia. La visita simmetrica visita nell'ordine: figlio sinistro, radice, figlio destro. Date le proprietà del BST, l'ordine è rispettato.
- **Passo induttivo** $h > 1$: Supponiamo che la proprietà valga per alberi di altezza inferiore a h. Per definizione, la visita simmetrica visita il sottoalbero sinistro L, poi la radice r e infine il sottoalbero destro R. Per ipotesi induttiva, L e R sono visitati in ordine crescente. Poiché ogni chiave in $L \leq \text{key}(r)$ e ogni chiave in $R \geq \text{key}(r)$, l'intera sequenza risultante sarà ordinata.

□

2.1.1 Operazioni

Search

L'algoritmo di ricerca ricalca il principio della ricerca binaria. Partendo dalla radice, si confronta la chiave cercata k con la chiave del nodo corrente v:

- Se $k = \text{key}(v)$, la ricerca termina con successo e ritorna.
- Se $k < \text{key}(v)$, la ricerca prosegue ricorsivamente nel sottoalbero **sinistro**.
- Se $k > \text{key}(v)$, la ricerca prosegue ricorsivamente nel sottoalbero **destro**.

Se si raggiunge un nodo nullo (`NULL`), la chiave non è presente nell'albero.

Algorithm 5: Ricerca in un BST

Input: Nodo radice u, chiave cercata k
Output: Il nodo con chiave k o `NULL`

```
while u ≠ NULL do
    if k == u.key then
        return u
    end
    if k < u.key then
        u ← u.left
    end
    else
        u ← u.right
    end
end
return NULL
```

Insert

L'obiettivo dell'inserimento è posizionare un nuovo nodo u come **foglia**, preservando le proprietà dell'albero. Il procedimento prevede i seguenti passi:

1. Si crea il nodo u con i relativi attributi (*elem*, *key*).
2. Si simula una procedura di *search(key(u))* per individuare il futuro nodo padre.
3. Una volta individuato il padre, si aggancia u come figlio destro o sinistro in modo da rispettare le proprietà dei BST.

Algorithm 6: Inserimento in un BST

Input: Nodo radice r, nuovo nodo u (già inizializzato)

```
y ← NULL                          // y sarà il padre di u
if r == NULL then
    r ← u                         // L'albero era vuoto
end
x ← r
while x ≠ NULL do
    y ← x
    if u.key < x.key then
        x ← x.left
    end
    else
        x ← x.right
    end
end
u.parent ← y
else if u.key < y.key then
    y.left ← u
end
else
    y.right ← u
end
```

Delete

L'operazione di cancellazione di un nodo v è la più complessa da implementare quindi prima di definire la cancellazione, è necessario introdurre alcune operazioni ausiliarie fondamentali.

- **Ricerca del minimo (findMin):** Partendo dal nodo radice, si segue ricorsivamente il puntatore **sinistro**. Il minimo è rappresentato dal primo nodo incontrato il cui puntatore sinistro è `NULL`.
- **Ricerca del successore (findSuccessor):** Il successore di un nodo v è il nodo con la minima chiave tra tutti i nodi aventi chiave maggiore di key(v). Si distinguono due casi:
  1. **Se v ha un sottoalbero destro:** il successore è semplicemente il valore minimo di tale sottoalbero (*findMin(v.right)*).
  2. **Se v non ha un figlio destro:** il successore si trova risalendo l'albero fino a individuare il primo antenato di v tale che v appartenga al suo sottoalbero **sinistro**.

Algorithm 7: Ricerca del minimo in un BST

Input: Nodo radice r
Output: Il nodo con chiave minima o `NULL`

```
if r == NULL then
    return NULL
end
x ← r
while x.left ≠ NULL do
    x ← x.left
end
return x
```

Algorithm 8: Ricerca del successore di un nodo in un BST

Input: Nodo v di cui cercare il successore
Output: Il nodo successore o `NULL`

```
if v.right ≠ NULL then
    return findMin(v.right)
end
u ← v
while u.parent ≠ NULL ∧ u.parent.right == u do
    u ← u.parent
end
return u.parent
```

Le operazioni di cancellazione richiede la gestione di tre scenari differenti:

- **Caso 1: v è una foglia.** Si rimuove il nodo aggiornando il corrispondente puntatore del padre a `NULL`.
- **Caso 2: v ha un solo figlio.** Si effettua un'operazione di "scavalcamento": il padre di v viene collegato direttamente all'unico figlio di v, rimuovendo v dalla struttura.
- **Caso 3: v ha due figli.** Non è possibile rimuovere v direttamente senza rompere la struttura dell'albero. Si procede quindi come segue:
  1. Si individua il **successore** di v (che si troverà necessariamente nel sottoalbero destro).

2. Si copia il contenuto (chiave ed elementi) del successore nel nodo v.
3. Si rimuove fisicamente il successore dall'albero. Poiché il successore è il minimo del sottoalbero destro, esso avrà al massimo un figlio (il destro), ricadendo quindi nel Caso 1 o nel Caso 2.

Algorithm 9: Cancellazione di un nodo in un BST

Input: Nodo radice r, nodo v da rimuovere

```
p ← v.parent
if v.left == NULL ∧ v.right == NULL then
    if p == NULL then
        r ← NULL
    end
    else if p.left == v then
        p.left ← NULL
    end
    else
        p.right ← NULL
    end
end
else if (v.left == NULL) xor (v.right == NULL) then
    if v.left ≠ NULL then
        child ← v.left
    end
    else
        child ← v.right
    end
    if p == NULL then
        r ← child
    end
    else if p.left == v then
        p.left ← child
    end
    else
        p.right ← child
    end
    if child ≠ NULL then
        child.parent ← p
    end
end
else
    s ← findSuccessor(v)
    v.key ← s.key
    v.elem ← s.elem
    Delete(r, s)
end
```

Note

Tutte le operazioni dipendono dall'altezza dell'albero per cui il loro costo è $O(h)$

2.2 Red-Black Trees

Lo standard per garantire l'efficienza di queste operazioni è imporre una *condizione di bilanciamento*. Tale vincolo assicura che l'altezza dell'albero rimanga logaritmica rispetto al numero di nodi ($h = O(\log n)$), evitando la degenerazione della struttura in una lista concatenata. Per farlo ad ogni nodo viene associata una funzione chiamata **rango** che assegna ad ogni nodo x un valore intero non negativo $rank(x)$ che deve soddisfare le seguenti proprietà fondamentali:

1. **Proprietà del Padre:** Se x è un nodo dotato di genitore $p(x)$, allora:
$$rank(x) \leq rank(p(x)) \leq rank(x) + 1$$
2. **Proprietà del Nonno:** Se x è un nodo dotato di un nonno $p^2(x)$, allora:
$$rank(x) < rank(p^2(x))$$
3. **Proprietà dei Nodi Esterni:** Se x è un nodo esterno (nodo con valore NULL), allora $rank(x) = 0$. Inoltre, se x possiede un genitore, allora $rank(p(x)) = 1$.

Come visto in [2] è possibile derivare una definizione alternativa, ma formalmente equivalente, basata sulla colorazione dei nodi. Si definisce un nodo x come **Nero** (*Black*) se $rank(p(x)) = rank(x) + 1$ (o se x è la radice dell'albero) e **Rosso** (*Red*) se $rank(p(x)) = rank(x)$.

Sotto questa formalizzazione, le condizioni di bilanciamento si traducono nelle proprietà canoniche dei **Red-Black Trees**:

1. Ogni nodo è marcato con il colore rosso o nero.
2. Un nodo rosso deve necessariamente avere un genitore e un figlio di colore nero.
3. Ogni nodo esterno è nero e tutti i cammini che congiungono un nodo x ai nodi esterni discendenti contengono il medesimo numero di nodi neri.
4. La radice dell'albero è nera

Note

$rank(v) = k$ è il numero di nodi neri che si incontrano dal nodo v, escluso, verso qualsiasi sua foglia

Lemma 2.2.1 *Un albero binario bilanciato con n nodi interni ha altezza $h \leq 2\log_2(n+1)$.*

Dimostrazione. Dimostriamo per induzione sul rango k che un nodo u con $rank(u) = k$ possiede un numero di discendenti interni $n(u) \geq 2^k - 1$, poi da qui determiniamo l'altezza dell'albero.

- **Caso base** ($k = 0$): per la Proprietà dei Nodi Esterni, un nodo con rango 0 è un nodo esterno. Esso non possiede discendenti interni, coerentemente con la formula: $2^0 - 1 = 0$.
- **Passo induttivo** ($k > 0$): Supponiamo che la proprietà sia vera per tutti i nodi con rango inferiore a k. Sia u un nodo con $rank(u) = k$ e siano v e w i suoi figli. Per la *Proprietà del Padre*, il rango di un figlio può essere k o $k - 1$. Tuttavia, analizziamo i due scenari possibili per un generico figlio v:

1. Se $rank(v) = k - 1$, per ipotesi induttiva v ha almeno $2^{k-1} - 1$ discendenti.
2. Se $rank(v) = k$, allora v deve essere necessariamente un nodo rosso. Per la *Proprietà del Nonno*, i figli di v (nipoti di u) devono avere rango strettamente inferiore a $rank(u)$, ovvero $k - 1$. In questo caso, applicando l'ipotesi induttiva ai due figli di v, il numero di discendenti di v è almeno $(2^{k-1} - 1) + (2^{k-1} - 1) + 1 = 2^k - 1$.

In entrambi i casi, ogni figlio di u garantisce almeno $2^{k-1} - 1$ discendenti. Sommando i contributi dei due figli di u e del nodo u:

$$n(u) \geq (2^{k-1} - 1) + (2^{k-1} - 1) + 1 = 2 \cdot 2^{k-1} - 1 = 2^k - 1$$

La proprietà è dunque dimostrata per ogni k.

Sia T un albero con n nodi interni e sia k il rango della sua radice. Dalla proprietà appena dimostrata sappiamo che:

$$n \geq 2^k - 1$$

Risistemando i termini

$$n + 1 \geq 2^k \implies \log_2(n + 1) \geq k$$

Dalla *Proprietà del Nonno*, sappiamo che lungo ogni cammino non possono esserci due nodi rossi consecutivi. Questo implica che l'altezza fisica h dell'albero può essere al massimo il doppio del rango della radice ($h \leq 2k$).

Sostituendo la relazione trovata per k:

$$h \leq 2k \leq \log_2(n + 1)$$

□

Le operazioni di inserimento (Insert) e di cancellazione (Delete) sono le medesime degli alberi binari tuttavia potrebbero rovinare il bilanciamento e violare le proprietà dell'albero quindi abbiamo bisogno di alcune operazioni ausiliarie che ripristino tutte le proprietà dei red-blak trees

Rotazioni e Ribilanciamento

Le rotazioni sono operazioni locali che modificano la struttura dell'albero preservando l'ordine simmetrico dei nodi (proprietà di ricerca). Un nodo v intuitivamente è **sbilanciato** se uno dei suoi sottoalberi è troppo alto rispetto agli alti.

Sia v il nodo sbilanciato possiamo distinguiamo quattro casi fondamentali in base alla posizione del sottoalbero T che causa lo sbilanciamento:

Caso Sinistro-Sinistro (SS): T è il figlio sinistro del figlio sinistro di v. Si risolve con una *rotazione semplice verso destra* su v.

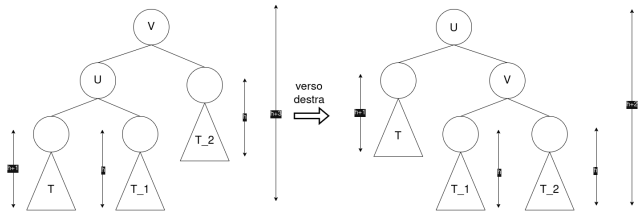

Figura 2.1: Rotazione semplice destra (Caso SS).

Caso Destro-Destro (DD): Speculare al caso SS. Si risolve con una *rotazione semplice verso sinistra* su v.

Caso Sinistro-Destro (SD): T è il figlio destro del figlio sinistro di v. Si risolve con una *rotazione doppia* (una rotazione verso sinistra sul figlio z, seguita da una verso destra su v).

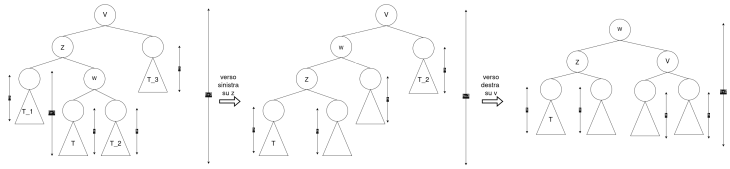

Figura 2.2: Rotazione doppia sinistra-destra (Caso SD).

Caso Destro-Sinistro (DS): Speculare al caso SD. Si risolve con una rotazione doppia destra-sinistra.

Lemma 2.2.2 *Sia v un nodo sbilanciato a seguito di un'operazione di inserimento o cancellazione. Esiste sempre una rotazione semplice o doppia in grado di ripristinare il bilanciamento del sottoalbero radicato in v.*

Dimostrazione. Si consideri il caso Sinistro-Sinistro (Fig. 2.1). Prima della rotazione, l'altezza del sottoalbero radicato in v è $h+3$. Dopo la rotazione semplice destra, il nodo u diventa la nuova radice locale. La nuova altezza sarà $h+2$
Ragionamento analogo può essere applicato alla rotazione doppia (Fig. 2.2), dove la sequenza di rotazioni trasforma un cammino critico di altezza $h+3$ in una struttura bilanciata di altezza $h+2$.
Poiché l'altezza viene ridotta esattamente del valore necessario a compensare lo sbilanciamento e l'ordinamento simmetrico è preservato, il nodo risulta correttamente ribilanciato. □

Insert

L'inserimento di un elemento in un *Red-Black Tree* (RBT) è un'operazione composta che integra la logica strutturale degli alberi binari di ricerca con un protocollo di ripristino delle proprietà cromatiche.

Sia v il nodo da inserire nell'albero T, si colora il nodo v di **rosso** e si seguendo l'algoritmo di inserimento standard per i BST.

Analisi della politica di colorazione La scelta di assegnare il colore rosso al nuovo nodo v è dettata dalla necessità di limitare le potenziali violazioni alle proprietà strutturali dell'RBT:

- **Invarianza dell'Altezza Nera (Proprietà 3):** L'assegnazione del colore rosso garantisce intrinsecamente il mantenimento della *Proprietà 3*, la quale impone che ogni cammino semplice da un nodo a una foglia discendente contenga il medesimo numero di nodi neri. Poiché v è rosso, il conteggio dei nodi neri (la cosiddetta *black-height*) per ogni cammino che attraversa v rimane inalterato rispetto alla configurazione precedente l'inserimento.
- **Vincoli di Adiacenza (Proprietà 2):** Le unice potenziali violazioni introdotte possono essere releative al fatto che il nodo padre (p(v)) è *NULL* o alla *Proprietà 2*, che vieta la presenza di due nodi rossi consecutivi. Tale conflitto si manifesta esclusivamente nell'eventualità in cui il padre del nodo inserito, $p(v)$, sia a sua volta di colore rosso, in questo caso va utilizzata la procedura fixInsert al fine ripristinare le proprietà dell'albero.

fixInsert

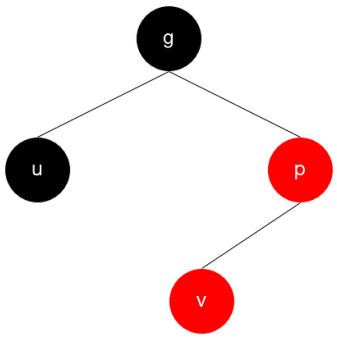

Figura 2.3: Caso del triangolo destro

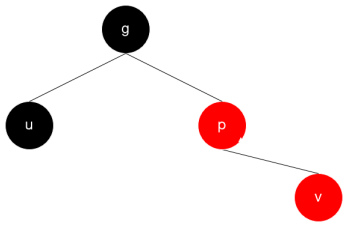

Figura 2.4: Caso della linea destra

La routine `fixInsert(v)` interviene quando il padre del nodo appena inserito, $p[v]$, è rosso o, violando la *Proprietà 2*. Il comportamento della procedura dipende dal colore e dalla posizione dello **zio** di v , indicato come u (fratello di $p[v]$).

Si distinguono i seguenti casi fondamentali:

1. **Caso della Radice:** Se $p[v]$ è `NULL`, significa che v è la radice dell'albero. In questo caso è sufficiente ricolorare v di nero per soddisfare tutte le proprietà.
2. **Ricolorazione (Zio Rosso):** Se lo zio u è rosso, il nonno g deve essere necessariamente nero (*Proprietà 2*). Si procede ricolorando $p[v]$ e u di nero, e g di rosso.
   - *Nota:* Poiché g è diventato rosso, la violazione potrebbe essersi spostata verso l'alto. La procedura viene quindi chiamata ricorsivamente su g.
3. **Caso del Triangolo :** Se lo zio u è nero (o `NULL`) e v è un figlio interno rispetto al nonno (es. v è figlio destro e $p[v]$ è figlio sinistro, come in Fig. 2.3), ci troviamo in una configurazione a "triangolo". Si applica una **rotazione semplice** sul padre $p[v]$ per ricondursi al caso della linea (Caso 4).
4. **Caso della Linea:** Se lo zio u è nero (o `NULL`) e v è un figlio esterno (es. entrambi figli sinistri, come in Fig. 2.4), i nodi sono allineati. Si applica una **rotazione semplice** sul nonno g, ricolorando il padre $p[v]$ di nero e il nonno g di rosso. Questa operazione risolve definitivamente la violazione senza alterare la *black-height*.

Remark 2.2.1 È importante notare che, mentre il Caso 2 può propagare la violazione fino alla radice con costo $O(\log n)$, i Casi 3 e 4 richiedono al massimo due rotazioni totali per terminare l'esecuzione della procedura.

Delete

L'operazione di rimozione di un nodo in un *Red-Black Tree* segue inizialmente la logica strutturale della cancellazione negli alberi binari di ricerca (BST). Tuttavia, le implicazioni cromatiche variano drasticamente in base al colore del nodo rimosso o del nodo che ne prende il posto:

- Se il nodo rimosso è **rosso**, le proprietà dell'albero rimangono invariate, poiché l'altezza nera dei cammini non subisce alterazioni e non si possono creare conflitti tra nodi rossi consecutivi.
- Se il nodo rimosso è **nero**, si verifica una violazione della *Proprietà 3*, poiché ogni cammino che attraversava quel nodo vedrà ridursi il proprio numero di nodi neri di un'unità.

Per ovviare a questa violazione, si introduce una procedura ausiliaria chiamata fixDelete che si occupa di ribilanciare la black-height

fixDelete

La procedura `fixDelete(x)` viene invocata qualora il nodo rimosso sia di colore nero, determinando una violazione della *Proprietà 3* (altezza nera). L'obiettivo della routine è eliminare tale riduzione risalendo l'albero o assorbendola tramite rotazioni.

Il comportamento dell'algoritmo è determinato dal colore del fratello f e dei relativi figli (nipoti di x): cl (figlio sinistro) e cr (figlio destro).

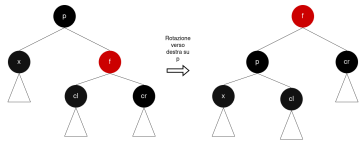

Figura 2.5: Caso del fratello Rosso

1. **Caso 1: Fratello f rosso.** Se f è rosso, il padre $p[x]$ deve essere necessariamente nero. Si esegue una rotazione (sinistra se x è figlio sinistro, destra se x è figlio destro come nel 2.5) su $p[x]$ e si scambiano i colori tra $p[x]$ e f. Questa operazione trasforma la configurazione in uno dei casi 2, 3 o 4, dove il nuovo fratello di x è nero.
2. **Caso 2: Fratello f e nipoti entrambi neri.** In questa situazione, vengono ricolorati x e da f di rosso ora controllo $p[x]$.
   - Se $p[x]$ era rosso, esso diventa semplicemente nero e la procedura termina.
   - Se $p[x]$ era già nero, allora la procedura `fixDelete` viene richiamata ricorsivamente verso l'alto su $p[x]$.

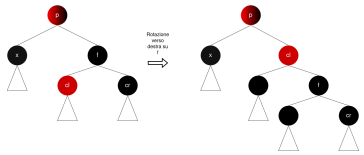

Figura 2.6: Caso del nipote vicino rosso

3. **Caso 3: Nipote "vicino" rosso (Zio nero).** Si verifica quando il figlio di f più prossimo a x è rosso (es. x è figlio sinistro e cl è rosso come nella figura 2.6). Si esegue una rotazione su f (direzione opposta a x) e si scambiano i colori tra f e il nipote rosso. Questa trasformazione riconduce l'albero al Caso 4.
4. **Caso 4: Nipote "lontano" rosso (Zio nero).** Si verifica quando il figlio di f più distante da x è rosso (es. x è figlio sinistro e cr è rosso come nella figura 2.7). Si esegue una rotazione su $p[x]$ verso x.
   - Il nodo f (nuova radice locale) assume il colore che aveva $p[x]$ prima della rotazione.
   - I nodi $p[x]$ e il nipote lontano rosso vengono colorati di nero.

   Questa operazione ripristina l'altezza nera.

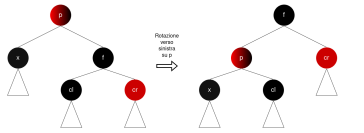

Figura 2.7: Caso del nipote lontano rosso

Note

Nei casi 3 e 4, il colore di p può essere sia rosso che nero.

Remark 2.2.2 Si osservi che, mentre il Caso 2 può propagare la violazione fino alla radice con costo $O(\log n)$, i Casi 3 e 4 garantiscono la terminazione della procedura dopo un numero costante di rotazioni.

Join

L'operazione di **Join** riceve in input due alberi Red-Black S_1 e S_2 e un nodo con chiave i, sotto l'ipotesi che:

$$\forall x \in S_1, \forall y \in S_2 : \text{key}(x) < i < \text{key}(y)$$

L'algoritmo si divide in due casi:

- **Caso** $rank(S_1) \geq rank(S_2)$: Poiché i e tutti gli elementi di S_2 sono maggiori di ogni chiave in S_1, l'obiettivo è inserire il blocco (i, S_2) all'interno di S_1.
  1. **Ricerca**: Si discende lungo il cammino dei figli destri di S_1 fino a individuare un nodo x tale che $rank(x) = rank(S_2)$.
  2. **Innesto**: Sia p il padre di x. Il nodo i viene inserito come nuovo figlio destro di p, x diventa il figlio sinistro di i, mentre la radice di S_2 diventa il figlio destro di i.
  3. **Colorazione**: Il nodo i viene colorato di **rosso**. Affinché la proprietà del rango sia preservata, le radici dei due sottoalberi x e S_2 vengono ricolorate di nere.
  4. **Ribilanciamento**: Viene invocata la procedura `fixInsert(i)` per risolvere eventuali violazioni della *Proprietà 2*
- **Caso** $rank(S_1) < rank(S_2)$: Il procedimento è simmetrico, ma questa volta si discende lungo il cammino dei figli sinistri di S_2 fino a trovare un nodo x con $rank(x) = rank(S_1)$. Il nodo i (colorato di rosso) sostituirà x, prendendo S_1 come figlio sinistro e x come figlio destro, poi si ricolora x e S_1 di **nero** e si richiama fixInsert(i)

Definition 2.2.1 *Il costo del* $join(S_1, i, S_2)$ *è* $O(log(n))$

Dimostrazione. La complessità temporale della procedura di $join(S_1, i, S_2)$ dipende dalla ricerca del nodo x, che richiede $|rank(S_1) - rank(S_2)|$, $O(1)$ per agganciare l'albero e la procedura `fixInsert`, richiede tempo proporzionale a quanto siamo discesi nell'albero $|rank(S_1) - rank(S_2)|$ Quindi, il costo complessivo è $O(|rank(S_1) - rank(S_2)| + 1) = O(log(n))$. □

Note

Il rank del nuovo albero che si formerà dopo join(S_1, i, S_2) sarà:

- Se rank(S_1) = rank(S_2), allora il nuovo rank sarà rank(S_1) + 1.
- Se rank$(S_1) \neq$ rank(S_2), allora il nuovo rank sarà max(rank(S_1), rank(S_2)).

Split

L'operazione di `split`(T, v) divide l'albero T in due alberi T_L e T_R rispetto alla chiave del nodo v. La procedura avviene in due fasi:

1. Si individua il nodo v e si inizializzano $T_L = v.left$ e $T_R = v.right$.
2. Si risale l'albero dal nodo v verso la radice. Per ogni nodo padre p nel cammino:

- Se v discende dal figlio sinistro di p, allora p e il sottoalbero $p.right$ appartengono alla parte destra. Si esegue: $T_R = \text{join}(T_R, p, p.right)$.
- Se v discende dal figlio destro di p, allora p e il sottoalbero $p.left$ appartengono alla parte sinistra. Si esegue: $T_L = \text{join}(p.left, p, T_L)$.

Definition 2.2.2 *Il costo dello split(T,v) è $O(log(n))$.*

Dimostrazione. Ci concentreremo sull'accolumatore T_L ma il ragionamento rimane invariato per T_R.
Ad ogni passo i se il cammino devia a destra eseguiremo $\text{Join}(S_1, n, T_{L,i})$ ossia uniremo quanto accumulato fino ad ora $T_{L,i}$ con perno il nodo sul cammino n e S_i che rappresenta il sottoalbero sinistro del nodo sul cammino, cosi da creare l'accumulatore $T_{L,i+1}$

claim

Ad ogni passo i della risalita $rank(S_i) \geq rank(T_{L,i})$

Dimostrazione. Per le proprietà del RB-Trees il la differenza di rank(o black-height) tra un nodo e i suoi nipoti è al massimo 1, supponiamo di avere

- Se il nodo n è nero i figli $n.left$ e $n.right$ avranno il medesimo rank mentre il padre avrà $rank(n.parent) = rank(n) + 1$. Di conseguenza il figlio sinistro di $n.parent$ ossia S_i dovrà avere se è rosso $rank(S_i) = rank(n.parent)$ mentre se è nero $rank(S_i) = rank(n) = rank(n.left)$
- Se il nodo n è rosso i figli $n.left$ e $n.right$ avranno rank minore di 1 mentre padre sara necessariamente nero e avra $rank(n.parent) = rank(n)$.Di conseguenza il suo figlio sinistro di $n.parent$ ossia S_i dovra avere se è rosso $rank(S_i) = rank(n.parent)$ mentre se è nero avrà lo stesso rank dei figli di n ossia $rank(S_i) = rank(n.left)$ e $rank(S_i) = rank(n.right)$

In entrambi i casi $rank(S_i)$ sarà o uguale o maggiore di 1 rispetto al rank dei nipoti
L'accumulato $T_{L,i}$ è il risultato delle join effettuate nei sottoalberi strettamente discendenti rispetto a S_i e siccome il rango diminuisce lungo il cammino e la funione join □

Siccome $rank(S_i) \geq rank(T_{L,i})$, avremo che $rank(T_{L,i+1}) = rank(S_i)$ oppure $rank(T_{L,i+1}) = rank(S_i) + 1$ $rank(T_{L,i+1}) \approx rank(S_i)$

Come abbiamo visto il costo di ogni singola operazione join_i è proporzionale alla differenza di altezza tra l'accumulatore corrente $T_{L,i}$ e il sottoalbero da unire S_i, più un costo costante per il collegamento:

$$\text{join}_{i-esimo} = O(rank(S_i) - rank(T_{L,i}) + 1) = O(h_{i+1} - h_i + 1)$$

Il costo totale dello `split` per T_L è dato dalla somma dei costi di tutte le operazioni di `join`:

$$\sum_{i=1}^{j} O(h_i - h_{i-1} + 1)$$

Espandendo la sommatoria:

$$O((h_1 - h_0) + (h_2 - h_1) + \cdots + (h_j - h_{j-1}) + \sum_{i=1}^{j} 1)$$

Notiamo che tutti i termini intermedi si cancellano a vicenda, lasciando solo la differenza tra l'altezza finale e quella iniziale, più il numero di passi j:

$$O(h_j - h_0 + j)$$

Poiché:

- $h_0 \geq 0$ è l'altezza del nodo di partenza;
- j è il numero di nodi sul cammino verso la radice, ovvero $O(\log n)$;

Concludiamo che la complessità è:

$$O(\log n - 0 + \log n) = O(\log n)$$

□

The harmony of the world is made manifest in Form and Number, and the heart and soul and all the poetry of Natural Philosophy are embodied in the concept of mathematical beauty.

— D'Arcy Wentworth Thompson

Bibliography

Here are the references in citation order.

[1] Monika Rauch Henzinger e Valerie King. «Randomized Dynamic Graph Algorithms with Polylogarithmic Time per Operation». In: *Journal of the ACM (JACM)* 46.4 (1999), pp. 502–516 (citato a pag. 1).

[2] Robert Endre Tarjan. *Data Structures and Network Algorithms*. Vol. 44. CBMS-NSF Regional Conference Series in Applied Mathematics. Philadelphia, PA: Society for Industrial e Applied Mathematics (SIAM), 1983 (citato a pag. 11).